Creating a Flash-Optimized Key-Value Store

This tutorial explains how to create a Flash-optimized key-value store using the Flash memory of the Portenta H7.

Author · Giampaolo Mancini, Sebastian Romero · Last revision · 01/25/2024

Overview

This tutorial explains how to create a Flash-optimized key-value store using the Flash memory of the Portenta H7. It builds on top of the **Flash In-Application Programming** tutorial.

Goals

In this tutorial you will learn how to use the Mbed OS TDBStore API to create a Key value store in the free space of the microcontroller's internal Flash.

Required Hardware and Software

- Portenta H7 (ABX00042), Portenta H7 Lite (ABX00045) or Portenta H7 Lite Connected (ABX00046)

Arduino IDE 2.0+ or Arduino CLI 0.13.0+

Instructions

MbedOS APIs for Flash Storage

The core software of Portenta H7 is based on the Mbed OS operating system, allowing developers to integrate the Arduino API with the APIs exposed by Mbed OS.

Mbed OS has a rich API for managing storage on different mediums, ranging from the small internal Flash memory of a microcontroller to external SecureDigital cards with large data storage space.

In this tutorial, you are going to save a value persistently inside the Flash memory. That allows to access that value even after a reset of the microcontroller. You will retrieve some information from a Flash block by using the **FlashIAPBlockDevice** and the **TDBStore** APIs. You will use the `FlashIAPBlockDevice` class to create a block device on the free space of the Flash and you will create a Key-Value Store in it using the `TDBStore` API.

Important: The TBStore API optimizes for access speed, reduce wearing of the flash and minimize storage overhead. TBStore is also resilient to power failures. If you want to use the Flash memory of the microcontroller, always prefer the TDBStore approach over a direct access to the FlashIAP block device.

1. The Basic Setup

Begin by plugging in your Portenta board to the computer using a USB-C® cable and open the Arduino IDE. If this is your first time running Arduino sketch files on the board, we suggest you check out how to set up the Portenta H7 for Arduino before you proceed.

2. Create the Structure of the Program

Let's program the Portenta with a sketch. You will also define a few helper functions in a supporting header file.

- Create a new sketch named `FlashKeyValue.ino`
- Create a new file named `FlashIAPLimits.h` to store the helper functions in a reusable file.

Note: Finished sketch its inside the tutorials library wrapper at: **Examples > Arduino_Pro_Tutorials > Creating a Flash-Optimized Key-Value Store > FlashKeyValueStore**

First let's add the helper functions to the `FlashIAPLimits.h` header. This will determine the available Flash limits to allocate the custom data.

```
/**
Helper functions for ca
**/

// Ensures that this fi
#pragma once

#include <Arduino.h>
#include <FlashIAP.h>
#include <FlashIAPBlock

using namespace mbed;

// A helper struct for
struct FlashIAPLimits {
  size_t flash_size;
  uint32_t start_addres
  uint32_t available_si
};

// Get the actual start
// considering the spac
FlashIAPLimits getFlash
{
  // Alignment lambdas
  auto align_down = [](
    return (((val) / si
  };
```

4. Make the Key Store Program

Go to `FlashKeyValue.ino` and include the libraries that you need from **MBED** and your header helper (`FlashIAPLimits.h`). The `getFlashIAPLimits()` function which is defined in the `FlashIAPLimits.h` header takes care of not overwriting data already stored on the Flash and aligns the

those calculated limits to create a block device and a `TDBStore` on top of them.

```
#include <FlashIAPBlockD
#include <TDBStore.h>

using namespace mbed;

// Get limits of the In
#include "FlashIAPLimits
auto iapLimits { getFlas

// Create a block device
FlashIAPBlockDevice bloc

// Create a key-value st
TDBStore store(&blockDev

// Dummy sketch stats da
struct SketchStats {
  uint32_t startupTime;
  uint32_t randomValue;
  uint32_t runCount;
};
```

In the `setup()` function at the beginning you will have to wait until the Serial port is ready and then print some info about the FlashIAP block device (`blockDevice`).

```
```

```
void setup()
{
  Serial.begin(115200);
  while (!Serial);

  //  Wait for terminal
  delay(1000);

  Serial.println("FlashI

  // Feed the random num
  srand(micros());

  // Initialize the flas
  blockDevice.init();
  Serial.print("FlashIAP
  Serial.println(blockDe
  Serial.print("FlashIAP
  Serial.println(blockDe
  Serial.print("FlashIAP
  Serial.println(blockDe
  Serial.print("FlashIAP
  Serial.println(blockDe
  // Deinitialize the de
  blockDevice.deinit();
```

After that, initialize the TDBstore (our storage space), set the key for the store data (`stats`), initialize the value that you will save `runCount` and declare an object to fetch the previous values (`previousStats`).

```
// Initialize the key-va
  Serial.print("Initiali
  auto result = store.in
  Serial.println(result
  if (result != MBED_SUC
    while (true); // Sto

  // An example key name
  const char statsKey[]

  // Keep track of the n
  uint32_t runCount { 0

  // Previous stats
  SketchStats previousSt
```

Now that you have everything ready, let's retrieve the previous values from the store and update the store with the new values.

```
// Get previous run sta
  Serial.println("Retri
  result = getSketchSta

  if (result == MBED_SU
    Serial.println("Pre
    Serial.print("\tSta
    Serial.println(prev
    Serial.print("\tRar
    Serial.println(prev
    Serial.print("\tRur

    Serial.println(prev

    runCount = previous

  } else if (result ==
    Serial.println("No
  } else {
    Serial.println("Err
    while (true);
  }

  // Update the stats a
  SketchStats currentSt
  result = setSketchSta

  if (result == MBED_SU
```

To finish the sketch, create `getSketchStats` and `setSketchStats` functions at the bottom of the sketch (after the `setup()` and `loop()`).

The `getSketchStats` function tries to retrieve the stats values stored in the Flash using the key `key` and returns them via the `stats` pointer parameter. Our `SketchStats` data struct is very simple and has a fixed size. You can therefore deserialize the buffer with a simple `memcpy`.

The `setSketchStats` function stores the `stats` data and assigns the key `key` to it. The key will be created in the key-value store if it does not exist yet.

```
// Retrieve SketchStats
int getSketchStats(cons
{
  // Retrieve key-value
  TDBStore::info_t info
  auto result = store.g

  if (result == MBED_ER
    return result;

  // Allocate space for
  uint8_t buffer[info.s
  size_t actual_size;

  // Get the value
  result = store.get(ke
  if (result != MBED_SU
    return result;

  memcpy(stats, buffer,
  return result;
}

// Store a SketchStats
int setSketchStats(cons
{
  return store.set(key,
}
```

5. Results

Upload the sketch, the output should be similar to the following:

```
FlashIAPBlockDevice + TD
FlashIAP block device si
FlashIAP block device re
FlashIAP block device pr
FlashIAP block device er
Initializing TDBStore: O
Retrieving Sketch Stats
Previous Stats
        Startup Time: 12
        Random Value: 15
        Run Count: 13
Sketch Stats updated
Current Stats
        Startup Time: 42
        Random Value: 21
        Run Count: 14
```

Note that the Flash memory will be **erased** when a new sketch is uploaded.

Push the reset button to restart the sketch. The values of the stats have been updated. `Previous Stats` which is retrieved from the key-value store now contains values from the previous execution.

Conclusion

You have learned how to use the available space in the Flash memory of the microcontroller to create a key-value store and use it to retrieve and store data.

It is not recommended to use the Flash of the microcontroller as the primary storage for data-intensive applications. It is best suited for read/write operations that are performed only once in a while such as storing and retrieving application configurations or persistent parameters.

Next Steps

- Learn how to retrieve a collection of keys using TDBStore iterators via `iterator_open` and `iterator_next`
- Learn how to create an incremental TDBStore set sequence via `set_start`, `set_add_data` and `set_finalize`
- Learn how to use the 16 MB QSPI Flash on the Portenta H7

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?